

Python Programming with Artificial Intelligence

Deep Learning

Huangxun Chen @IoT&AI Thrust



Deep Learning and PyTorch

- the need for deep learning frameworks

Tensors in PyTorch

- attributes, creating, reshaping, moving tensors to GPU
- tensor Operations: element-wise, broadcasting, in-place, indexing/slicing, concatenation, stack, split, aggregation, linear algebra

Autograd & Model Building

- computation graph, loss functions, backpropagation and autograd, optimizer
- building models with nn.Module
- model.train(), model.eval(), torch.no_grad()
- saving and loading models

Dataset & Dataloader

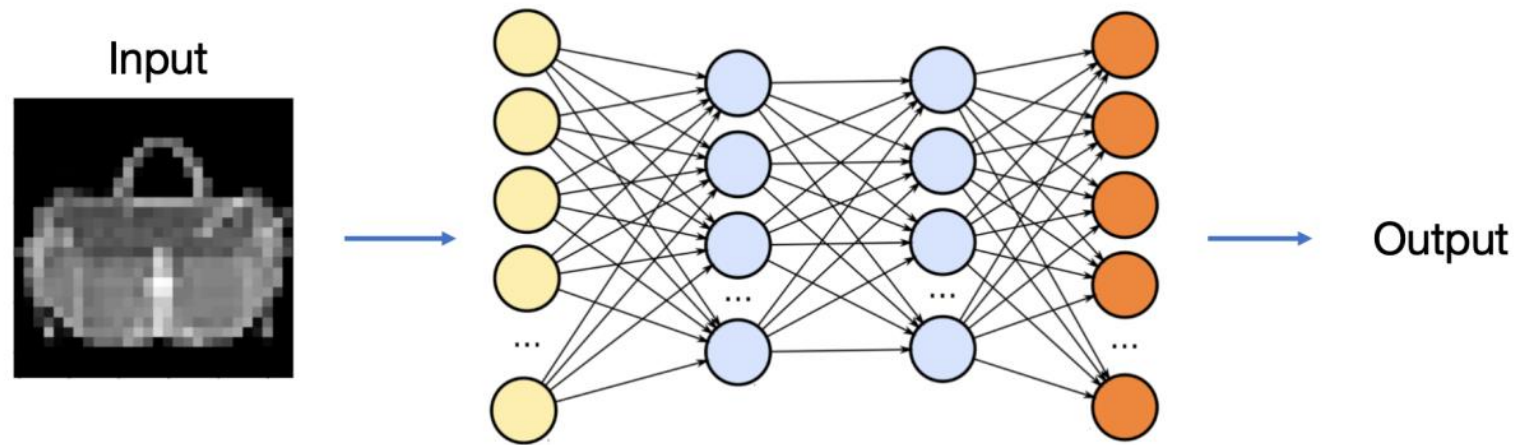
- build-in dataset, custom dataset



Deep Learning and PyTorch

What is Deep Learning?

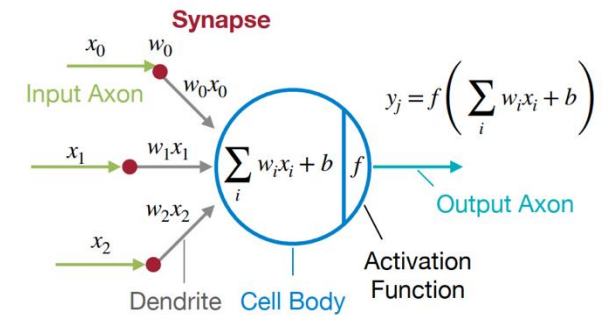
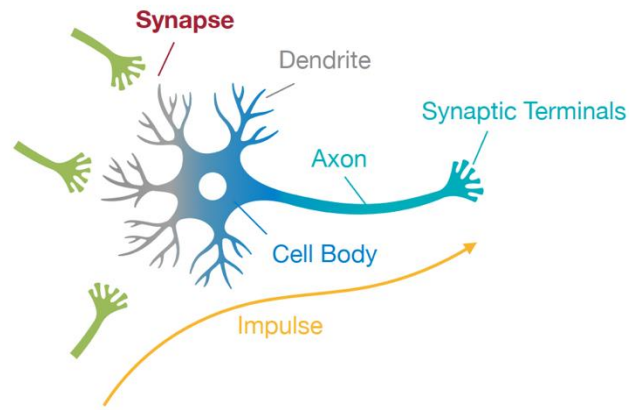
A subset of machine learning methods based on Artificial Neural Networks (ANNs) with representation learning.



Artificial Neural Networks (ANNs)

A machine learning model inspired by the structure and function of biological neural networks in animal brains.

- Units/Nodes: Mimic neurons.
- Edges/Weights: Mimic synapses.



Information is passed along edges, aggregated, and transformed

$$z = \sigma(w^T x + b)$$

■ ■ ■ Why is Deep Learning So Powerful?

Deep learning has amazing expressiveness in modeling complicated data and tasks.

Universal Approximation Theorem (UAT):

An ANN with at least one hidden layer and a non-linear activation function can approximate any continuous function to an arbitrary level of accuracy.

Deep learning forms the foundation of modern AI applications.

Examples of Deep Learning Architectures

Models vary based on how the hidden units are connected.

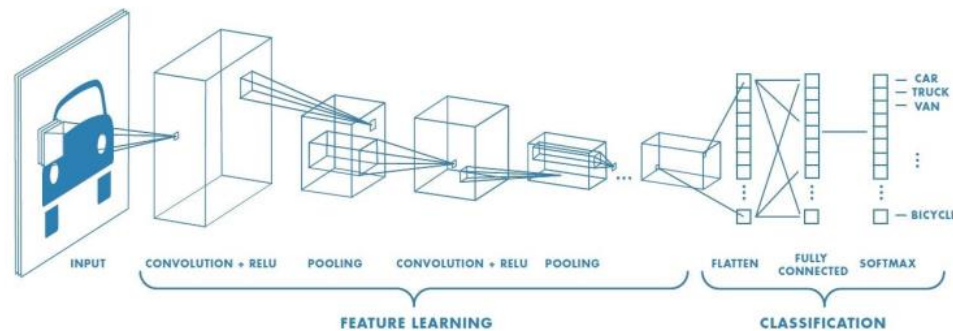


Figure: CNN for image classification

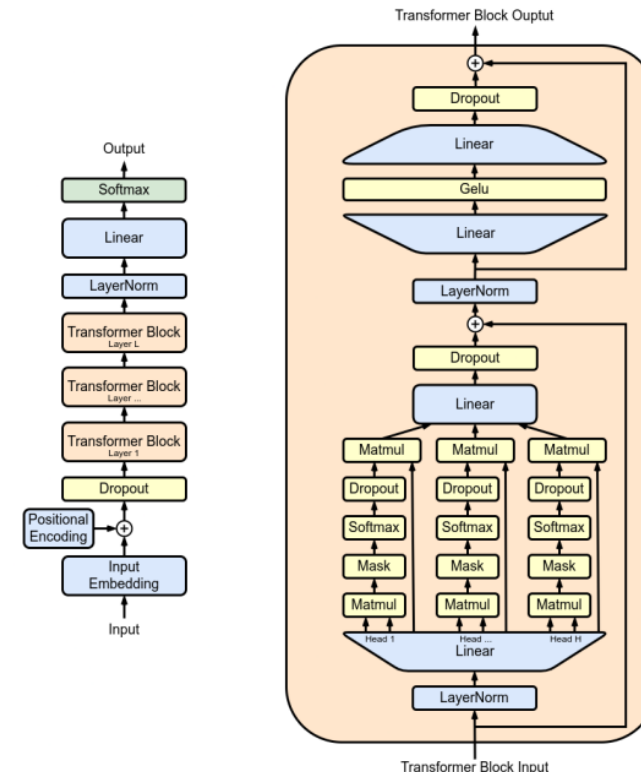


Figure: Transformer for texts (e.g., GPT)

The Need for Deep Learning Frameworks

Building a deep learning model involves:

- Customizing the structure (units and edges).
- Adapting data to the network's input.
- Training the model with a suitable optimizer.
- (The model can be very large) – Accelerating the massive computations.

Deep Learning Frameworks

Deep Learning Framework is a library that provides tools to assist developers in building and training deep learning models:

- GPU Support
- Automatic Differentiation (Autograd)
- Built-in, optimized network layers and loss functions



Why PyTorch?

While there are many other frameworks (TensorFlow, Keras), PyTorch is the most popular for research and teaching.

- Fast GPU support.
- The interface is similar to NumPy.
- Rich ecosystem of built-in algorithms, models, and datasets.
- Flexible 'nn.Module' for easy encapsulation of models.
- Dynamic Computation Graph (Autograd): Easy to code and debug.
- Huge community and rich third-party extension libraries (e.g., Hugging Face)

PyTorch Build	Stable (2.9.0)		Preview (Nightly)	
Your OS	Linux	Mac		Windows
Package	Pip	LibTorch		Source
Language	Python		C++ / Java	
Compute Platform	CUDA 12.6	CUDA 12.8	CUDA 13.0	ROCm 6.4 CPU
Run this Command:	pip3 install torch torchvision			



Tensors in PyTorch

What is a PyTorch Tensor?

A torch.Tensor is the basic operand in PyTorch.

Numpy on GPUs: A PyTorch Tensor is a multi-dimensional array, highly similar to a NumPy ndarray

- NumPy ndarray:
 - Runs only on the CPU.
 - Does not track operations.
- PyTorch 'Tensor:
 - Can run on either the CPU or a GPU.
 - Can automatically build a graph to calculate gradients (Autograd).

■ Importing and Creating Tensors

The standard import is 'torch'

```
import torch
```

```
# Create a 1D tensor (a vector) from 0 to 11  
x = torch.arange(12)  
print(x)
```

Output

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

Creating Tensors: Initializers

Create tensors with specific values, like all zeros or all ones

```
# A tensor of all zeros
z = torch.zeros((2, 3, 4))
print("Zeros shape:", z.shape)

# A tensor of all ones
o = torch.ones((3, 4))
print("Ones:\n", o)
```

Initialize model parameters with random values.

```
# Random numbers from a standard normal distribution
# (mean=0, std=1)
r_norm = torch.randn(3, 4)
print("randn:\n", r_norm)

# Random numbers from a uniform distribution [0, 1)
r_unif = torch.rand(2, 2)
print("\nrand:\n", r_unif)
```

Creating Tensors: From Python Lists & NumPy Array

We can create a tensor directly from a Python list or NumPy array

```
# Create from a nested list
T_list = torch.tensor([[2, 1, 4, 3],
                       [1, 2, 3, 4],
                       [4, 3, 2, 1]])

print(T_list)
```

```
# Create from a NumPy array
import numpy as np
A = np.array([[1, 2], [3, 4]])
T_np = torch.tensor(A)
print(T_np)
```

torch.tensor()

Bridge to NumPy

Converting between PyTorch and NumPy is easy, and they share the same underlying memory (on CPU).

```
# To NumPy
a_torch = torch.ones(5)
b_numpy = a_torch.numpy()
print(f"NumPy array: {b_numpy}")

# From NumPy
a_numpy = np.ones(5)
b_torch = torch.from_numpy(a_numpy)
print(f"PyTorch tensor: {b_torch}")

# They share memory!
a_torch[0] = 99
print(f"NumPy array changed: {b_numpy}")
```

Output

```
NumPy array: [1. 1. 1. 1. 1.]
PyTorch tensor: tensor([1., 1., 1., 1., 1.], dtype=torch.float64)
NumPy array changed: [99. 1. 1. 1. 1.]
```

Tensor Attributes: '.shape' and '.numel()'

We can inspect the tensor's properties

```
x = torch.arange(12)

# .shape tells us the size of each dimension
print(x.shape)

# .numel() tells us the total number of elements
print(x.numel())
```

Output

```
torch.Size([12])
12
```

Changing Shape: '.reshape()'

We can change a tensor's shape as long as the total number of elements remains the same.

```
x = torch.arange(12)

# Reshape x into a 3x4 matrix (2D Tensor)
X = x.reshape(3, 4)
print(X)
```

Output

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

Flexible Reshaping with '-1'

We can use '-1' to tell PyTorch to automatically infer one dimension.

```
x = torch.arange(12)

# Automatically infer the number of rows
X1 = x.reshape(-1, 4)
print("X1 shape:", X1.shape)

# Automatically infer the number of columns
X2 = x.reshape(3, -1)
print("X2 shape:", X2.shape)
```

Output

```
X1 shape: torch.Size([3, 4])
X2 shape: torch.Size([3, 4])
```

Tensor Attributes: 'dtype' and 'device'

Tensors have a data type and are stored on a device (CPU or GPU).

```
tensor = torch.rand(3, 4)

print(f"Shape of tensor: {tensor.shape}")
print(f"Datatype of tensor: {tensor.dtype}")
print(f"Device tensor is stored on: {tensor.device}")
```

Output

```
Shape of tensor: torch.Size([3, 4])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu
```

Moving Tensors to GPU

To perform fast computations, we can move our data and model to the GPU.

```
# Check if a GPU is available
if torch.cuda.is_available():
    # 1. Define the device
    device = torch.device("cuda")

    # 2. Create a tensor on the CPU (default)
    x_cpu = torch.rand(2, 2)
    print("x on CPU:\n", x_cpu)

    # 3. Move the tensor to the GPU
    x_gpu = x_cpu.to(device)
    print("\nx on GPU:\n", x_gpu)

    # Operations on x_gpu will now be executed on the GPU
else:
    print("No GPU available, running on CPU.")
```

Tensor Operations: Element-wise

These apply an operation to each element independently.

```
x = torch.tensor([1.0, 2, 4, 8])
y = torch.tensor([2, 2, 2, 2])

print(x + y)
print(x * y)
print(x ** y) # Exponentiation
```

Tensor Operations: Broadcasting

PyTorch automatically expands dimensions of size 1 to match shapes

```
a = torch.arange(3).reshape((3, 1)) # Shape [3, 1]
b = torch.arange(2).reshape((1, 2)) # Shape [1, 2]

# a (shape 3,1) and b (shape 1,2)
# a is stretched 2x along dim 1
# b is stretched 3x along dim 0
c = a + b
print(c)
print(c.shape)
```

Output

```
tensor([[0, 1],
        [1, 2],
        [2, 3]])
torch.Size([3, 2])
```

In-place Operations

Standard ops create new tensors. In-place ops modify existing tensors.

```
X = torch.rand(2, 2)
Y = torch.rand(2, 2)

# Standard (out-of-place)
id_before = id(X)
X = X + Y
id_after = id(X)
print("New tensor created:", id_before == id_after)

# In-place (more efficient)
id_before = id(X)
X += Y
id_after = id(X)
print("In-place operation:", id_before == id_after)
```

Output

```
New tensor created: False
In-place operation: True
```

Tensor Operations: Indexing and Slicing

Indexing and Slicing work just like in NumPy

```
X = torch.arange(12).reshape((3, 4))
print("Original:\n", X)
# Get row 0
print("Row 0:", X[0])
# Get column 1
print("Column 1:", X[:, 1])
# Get a single element
print("Element [1, 2]:", X[1, 2])
# Set a slice
X[0:2, 0:2] = 99
print("\nModified:\n", X)
```

Original:

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

Row 0: tensor([0, 1, 2, 3])

Column 1: tensor([1, 5, 9])

Element [1, 2]: tensor(6)

Modified:

```
tensor([[99, 99,  2,  3],
        [99, 99,  6,  7],
        [ 8,  9, 10, 11]])
```

Tensor Operations: Concatenation

We can join tensors together using `torch.cat()`

```
X = torch.arange(6).reshape((2, 3))
Y = torch.ones((2, 3))

# Concatenate along dim=0 (rows)
cat_0 = torch.cat((X, Y), dim=0)
print("Cat dim=0 shape:", cat_0.shape)

# Concatenate along dim=1 (columns)
cat_1 = torch.cat((X, Y), dim=1)
print("Cat dim=1 shape:", cat_1.shape)
```

Output

```
Cat dim=0 shape: torch.Size([4, 3])
Cat dim=1 shape: torch.Size([2, 6])
```

Tensor Operations: Stack

torch.stack joins a sequence of tensors along a new dimension.
All tensors in the sequence must be the same size.

```
import torch
a = torch.rand(2, 3)
b = torch.rand(2, 3)

# Stack along new dim=1
c = torch.stack((a, b), dim=1)
print(c.shape)
```

Output

```
torch.Size([2, 2, 3])
```

- a.shape is [2, 3]
- b.shape is [2, 3]
- We stack 2 tensors.
- New dimension at dim=1.
- Resulting shape: [2, 2, 3]

Tensor Operations: Split

`torch.split` splits a tensor into chunks along an existing dimension.
We can specify the size of each chunk.

```
import torch
a = torch.rand(4, 6, 10)

# Split along dim=1 into
# chunks of size 2
lst = a.split(split_size=2, dim=1)

print(len(lst), lst[0].shape)
```

Output

```
3 torch.Size([4, 2, 10])
```

- Input shape: [4, 6, 10]
- Split dim=1 (size 6)
- Chunk size is 2
- $6 / 2 = 3$ chunks
- `len(lst)` is 3
- Each chunk's shape: [4, 2, 10]

Tensor Operations: Aggregations (sum, mean, max)

“aggregate” a tensor along a dimension.

```
a = torch.rand(4, 6, 10)

# Sum all elements
print("Total sum:", a.sum())

# Sum along a specific dimension (dim=1)
# Shape changes from [4, 6, 10] -> [4, 10]
print("Sum(dim=1) shape:", a.sum(dim=1).shape)

# Get the mean and max value
print("Mean:", a.mean(), "Max:", a.max())
```

Output

```
Total sum: tensor(112.8433)
Sum(dim=1) shape: torch.Size([4, 10])
Mean: tensor(0.4702) Max: tensor(0.9843)
```

Linear Algebra: torch.mv, torch.mm, @

For linear algebra, we use specific functions.

```
A = torch.rand(3, 4)
B = torch.rand(4, 5)
x = torch.rand(4)

# Matrix-Vector Product
mv_prod = torch.mv(A, x) # Shape [3]

# Matrix-Matrix Product
mm_prod = torch.mm(A, B) # Shape [3, 5]

# The @ operator is a convenient shortcut!
mm_prod_at = A @ B
print(torch.allclose(mm_prod, mm_prod_at))
```



Autograd & Model Building

Why Autograd? The need for gradients

- Our goal is to train a model, i.e., find the best parameters (weights & biases).
- We use **Gradient Descent** to minimize a Loss function.
- To do this, we must calculate the gradient of the Loss with respect to every model parameter $\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial w_3}, \dots$

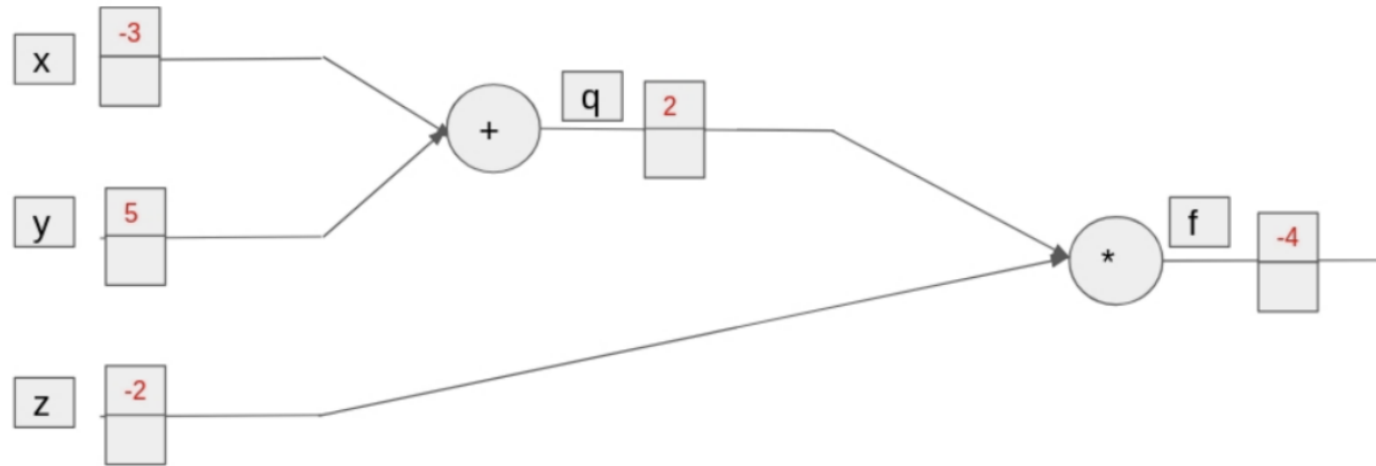
Challenge: Modern deep neural networks can have millions or billions of parameters. Manually deriving the gradient formulas using the chain rule is practically impossible, extremely complex, and error-prone.

Solution:

- An **Autograd** (Automatic Differentiation) system automatically computes all these gradients for us.
- To do this, it first needs to build a “**Computation Graph**” to record all operations.

Computation Graph

- Tensor operations create a computation graph
- **Forward Pass:** Input data moves forward through hidden layers until reaching the output.
 - PyTorch tracks all operations performed on tensors and their inputs, storing them in a computation graph.



Example: $q = x + y$, $f = q * z$

Neural Network with Tensor Operations

We can create a computation graph for a neural network using tensor operations.

```
import torch
X = torch.rand(10)
w1 = torch.rand(10, 20)
w2 = torch.rand(20, 20)
w3 = torch.rand(20, 4)

h1 = torch.tanh(X @ w1)
h2 = torch.tanh(h1 @ w2)
y_pred = torch.softmax(h2 @ w3, dim=0)
y_pred
```

Example: a 4-layer network [10, 20, 20, 4]

With proper parameters, UAT suggests that we can get ground truth y for any x

Problem: how to find these parameters?

- Specify Loss Function
- Optimization via Gradient Descent
 - Backpropagation
 - PyTorch Autograd
- Specify Optimizer

Loss Functions

- Measures how the prediction deviated from the target value.
- This is the value to be optimized (minimized) during model training.
- Example: Mean Squared Error (MSE)

$$\min_{\theta} \frac{1}{n} \sum_i^n (y_i - f_{\theta}(x_i))^2$$

- Example: Cross Entropy Loss

$$\min_{\theta} -\frac{1}{n} \sum_i^n [y_i \log(f_{\theta}(x_i)) + (1 - y_i) \log(1 - f_{\theta}(x_i))]$$

Loss Functions

- PyTorch provides various built-in loss functions.
- **How they work:** They receive two tensors (e.g., predictions & targets), get an element-wise loss, and then aggregate them into a single scalar value.

nn.L1Loss	nn.GaussianNLLLoss	nn.HingeEmbeddingLoss	nn.MultiLabelSoftMarginLoss
nn.MSELoss	nn.KLDivLoss	nn.MultiLabelMarginLoss	nn.CosineEmbeddingLoss
nn.CrossEntropyLoss	nn.BCELoss	nn.HuberLoss	nn.MultiMarginLoss
nn.CTCLoss	nn.BCEWithLogitsLoss	nn.SmoothL1Loss	nn.TripletMarginLoss
nn.NLLLoss	nn.MarginRankingLoss	nn.SoftMarginLoss	nn.TripletMarginWithDistanceLoss

```
from torch.nn import CrossEntropyLoss
y_true = torch.tensor(1)
# y_pred from previous slide (example)
# tensor([0.1694, 0.4777, 0.0183, 0.3346])
loss_f = CrossEntropyLoss()
loss = loss_f(y_pred, y_true)
```

Optimization via Gradient Descent

- **Optimization:** The procedure of finding the parameters that minimize the loss function.
- **Gradient Descent:** We update model parameters towards the gradient direction.
- The key is to obtain the **gradient** (i.e., partial derivative) with respect to all the parameters.

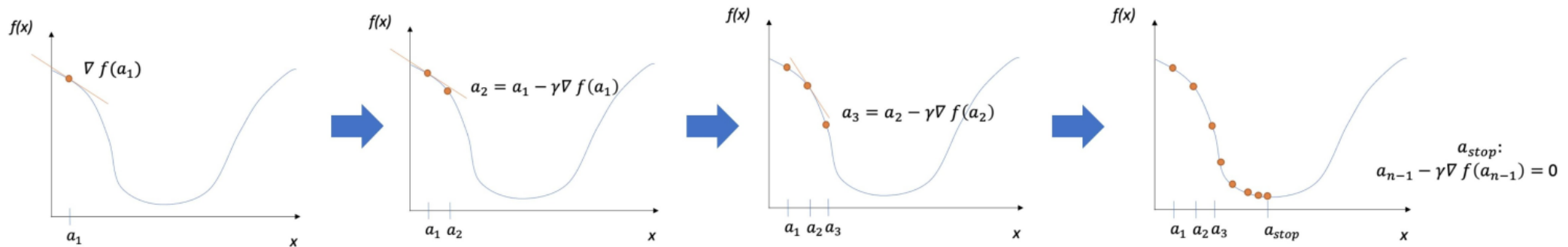


Figure: Gradient descent iteratively updates parameters to find a minimum.

Backpropagation

- Backward Pass: The gradient for the loss is propagated backward through the computation graph. Each variable/parameter in the graph obtains its gradient

Sum:

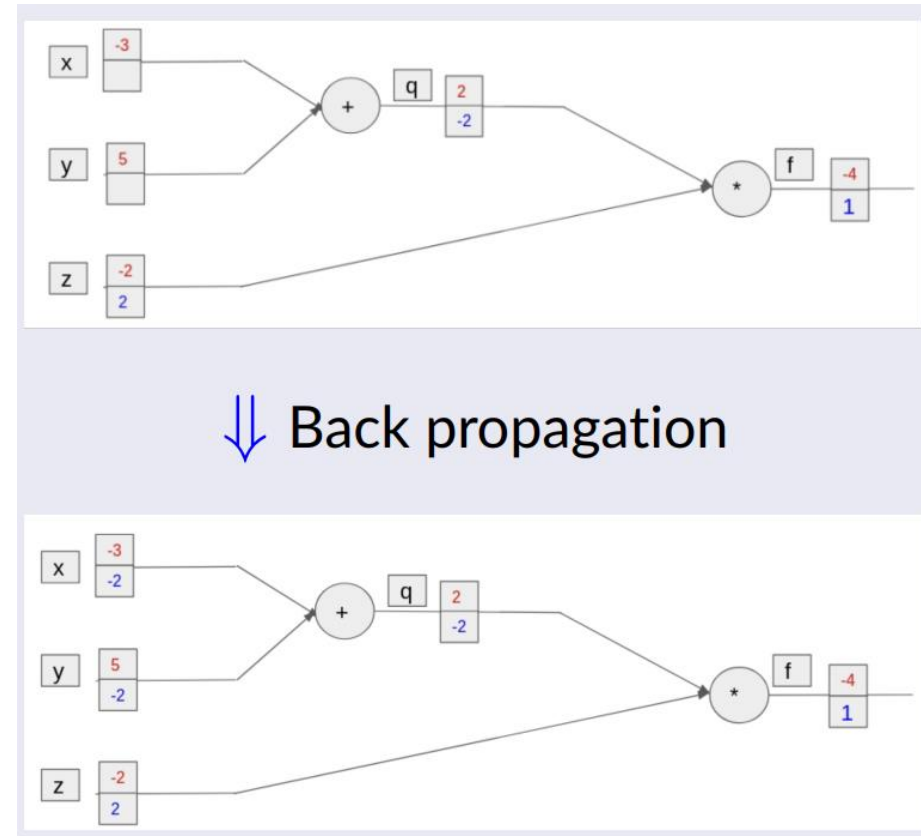
$$\frac{d}{dx}(f(x) + g(x)) = \frac{d}{dx}f(x) + \frac{d}{dx}g(x)$$

Production:

$$\frac{d}{dx}(f(x)g(x)) = f(x)\frac{d}{dx}g(x) + g(x)\frac{d}{dx}f(x)$$

Chain rule:

$$\frac{d}{dx}f(g(x)) = \frac{df}{dg} \cdot \frac{dg}{dx}$$



Example: $q = x + y$, $f = q * z$

PyTorch Autograd: Automatic Differentiation

- `backward()` performs automatic differentiation.
- It propagates gradients backward through the computational graph, adds the gradient to the `.grad` attribute of each tensor with `requires_grad=True`.

```
import torch
x = torch.tensor(-3., requires_grad=True)
y = torch.tensor(5., requires_grad=True)
z = torch.tensor(-2., requires_grad=True)
q = x + y
f = q * z
f.backward() # This computes all gradients

print("Gradient of z is: " + str(z.grad))
print("Gradient of y is: " + str(y.grad))
print("Gradient of x is: " + str(x.grad))
```

```
Gradient of z is: tensor(2.)
Gradient of y is: tensor(-2.)
Gradient of x is: tensor(-2.)
```

Autograd on Neural Network Parameters

- We set requires_grad=True for our model weights.
- Calling `loss.backward()` will compute all their gradients.

```
import torch
from torch.nn import CrossEntropyLoss
X = torch.rand(10)
y_true = torch.tensor(1)
# Set requires_grad=True for weights
w1 = torch.rand(10, 20, requires_grad=True)
w2 = torch.rand(20, 20, requires_grad=True)
w3 = torch.rand(20, 4, requires_grad=True)

# (Forward pass...)
h1 = torch.tanh(X @ w1)
h2 = torch.tanh(h1 @ w2)
y_pred = torch.softmax(h2 @ w3, dim=0)
loss_f = CrossEntropyLoss()
loss = loss_f(y_pred, y_true)
loss.backward() # Compute gradients

# Gradients are now populated
# w1.grad, w2.grad, w3.grad
```

Autograd on Neural Network Parameters

- A more recommended way is to use `nn.Parameter`.

```
from torch import nn
# nn.Parameter automatically
# sets requires_grad=True and
# signals this is a model parameter
#
w1 = nn.Parameter(torch.rand(10, 20))
w2 = nn.Parameter(torch.rand(20, 20))
w3 = nn.Parameter(torch.rand(10, 4))
# ...
```

- `torch.optim` is a package with various optimization algorithms (SGD, Adam, RMSprop, etc.).
- One example: Stochastic Gradient Descent (SGD)
 - We initialize an optimizer, passing it the parameters to update (params) and a learning rate (lr) γ .
 - The update rule (for SGD) is: $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$
 - `optimizer.step()` updates parameters using their `.grad` attribute.

```
from torch.optim import SGD

# Assume w1, w2, w3 are parameters
# (from previous slide)
# loss.backward() has been called

# 1. Initialize the optimizer
optimizer = SGD([w1, w2, w3], lr=0.01)

# 2. Tell the optimizer to update the parameters
optimizer.step()

# w1, w2, w3 have now been modified
```

Putting It All Together: The Training Loop

We repeat the process for multiple steps in a loop

```
# (Imports and w1, w2, w3 setup)
optimizer = SGD([w1, w2, w3], lr=0.01)
n_steps = 5

for step in range(n_steps):
    # 1. Forward pass
    h1 = torch.tanh(X @ w1)
    h2 = torch.tanh(h1 @ w2)
    y_pred = torch.softmax(h2 @ w3, dim=0)

    # 2. Calculate loss
    loss_f = CrossEntropyLoss()
    loss = loss_f(y_pred, y_true)

    # 3. Reset gradients
    optimizer.zero_grad()

    # 4. Backward pass (Autograd)
    loss.backward()

    # 5. Update parameters
    optimizer.step()
```

The core pattern for ALL PyTorch training

```
# Inside your loop...

# 1. Reset gradients
optimizer.zero_grad()

# 2. Calculate loss and compute new gradients
loss = ...
loss.backward()

# 3. Update the model's parameters
optimizer.step()
```



We must call `optimizer.zero_grad()` because gradients accumulate by default.

PyTorch uses a **Dynamic Computation Graph**, so the graph is re-created on each forward pass.

In-Class Practice

Your goal is to build, train, and evaluate a neural network that learns a pre-defined function.

- Step 1: Generate Data. Generate a dataset (e.g., 100 samples) from the function:

$$y = 3x_1 + 4x_2 - x_3^2$$

- Step 2: Construct Model. Construct a neural network to predict y given x . You will need to customize the depth (layers) and width (neurons).
- Step 3: Train Model. Train your network on the data. Monitor the training loss and observe how it changes over time.
- Step 4: Evaluate Model. Generate a new set of test data and evaluate your model's performance on it.

Note:

- Your input tensor X should have the shape (100, 3).
- Your target tensor y should have the shape (100,).
- Loss functions (like `nn.MSELoss`) can be applied directly to tensors, not just scalars.



Building Models with `nn.Module`

The Problem with Raw Tensors

Managing lists of raw weight tensors ('w1', 'w2', 'w3'...) is messy and error-prone.

- We have to pass '[w1, w2, w3]' to the optimizer.
- We have to write the forward pass logic 'h1 = torch.tanh(X @ w1)' in the main loop.
- It's not a reusable, self-contained "model".

Solution: nn.Module

- PyTorch provides a base class, nn.Module, that helps us solve this problem.

Torch Modules: The nn.Module Class

- A **high-level abstraction** of neural networks, offering easy-to-use interfaces for defining and training sophisticated models.
- It **packs weights (parameters) and forward pass operations** into a single, unified module.

Benefits:

- Modular: Can be nested (modules within modules).
- Reusable: Define once, use many times.
- Optimized: Includes optimized implementations of common structures.
- Easy GPU Acceleration: Just call `.to(device)`.

Torch Modules: The nn.Module Class

Two Kinds:

- Customized modules (what we will learn to build)
- Built-in modules (e.g., nn.Linear, nn.Conv2d)

Customize Complex Modules

We create a class that inherits from `nn.Module`.

```
import torch.nn as nn
import torch.nn.functional as F # Contains activation functions

class MySimpleNet(nn.Module):

    # 1. Define layers in __init__
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__() # Always call this first!

        # Define the layers. PyTorch will automatically
        # register these as parameters to be trained.
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.layer2 = nn.Linear(hidden_size, output_size)

    # 2. Define the forward pass logic
    def forward(self, x):
        # x -> layer1 -> relu -> layer2
        x = self.layer1(x)
        x = F.relu(x)
        x = self.layer2(x)
        return x
```

Customize Complex Modules

Now, using the module is clean and simple.

```
# 1. Instantiate the model
model = MySimpleNet(input_size=10, hidden_size=20, output_size=2)
model.to(device) # Move all parameters to the GPU

# 2. Create an optimizer that *automatically* finds all parameters
# We just pass in model.parameters()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# 3. Create dummy data
inputs = torch.rand(64, 10).to(device) # A batch of 64
labels = torch.randint(0, 2, (64,)).to(device)

# --- The training loop is now much cleaner! ---
optimizer.zero_grad()
predictions = model(inputs) # Calls the forward() method
loss = nn.CrossEntropyLoss()(predictions, labels)
loss.backward()
optimizer.step()

print("Training step complete!")
```

The nn.Module Magic

When we inherit from `nn.Module` and assign a layer as an attribute (e.g., `self.layer1 = nn.Linear(...)`), PyTorch:

- Automatically adds the layer's weights and biases to the model's parameter list.
- This is why `model.parameters()` just works.
- This is also why `model.to(device)` moves all sub-modules to the GPU.

Alternative: `nn.Sequential`

For simple, feed-forward models, `nn.Sequential` is a fast way to build a model without writing a class.

```
input_size = 10
hidden_size = 20
output_size = 2

model_seq = nn.Sequential(
    nn.Linear(input_size, hidden_size),
    nn.ReLU(),
    nn.Linear(hidden_size, output_size)
)

model_seq.to(device)
print(model_seq)
```

```
Sequential(
  (0): Linear(in_features=10, out_features=20, bias=True)
  (1): ReLU()
  (2): Linear(in_features=20, out_features=2, bias=True)
)
```

Built-in Modules

PyTorch provides optimized, pre-built layers for almost everything.

Module Type	Examples
Linear	<code>nn.Linear</code>
Convolutional	<code>nn.Conv2d</code> , <code>nn.Conv1d</code>
Recurrent	<code>nn.RNN</code> , <code>nn.LSTM</code> , <code>nn.GRU</code>
Transformer	<code>nn.Transformer</code> , <code>nn.TransformerEncoderLayer</code>
Activation	<code>nn.ReLU</code> , <code>nn.Sigmoid</code> , <code>nn.Tanh</code> , <code>nn.Softmax</code>
Pooling	<code>nn.MaxPool2d</code> , <code>nn.AvgPool2d</code> , <code>nn.AdaptiveAvgPool2d</code>
Normalization	<code>nn.BatchNorm2d</code> , <code>nn.LayerNorm</code>
Dropout	<code>nn.Dropout</code>

`model.train()` vs. `model.eval()`

Some layers, like `nn.Dropout` and `nn.BatchNorm2d`, behave differently during training and evaluation.

- `model.train()`: (Default) Tells all modules (like `Dropout`) to be in “training” mode. We should call this before your training loop.
- `model.eval()`: Tells all modules to be in “evaluation” mode. `Dropout` is turned off, `BatchNorm` uses running statistics. We **must call this before evaluating our model on the test set**.

When we run evaluation, we don't need to build the computation graph or calculate gradients.

Wrapping our evaluation code in `with torch.no_grad()`:

- Disables gradient calculation.
- Speeds up computation.
- Saves a lot of memory

```
model.eval() # Set model to evaluation mode
with torch.no_grad(): # Turn off autograd
    for inputs, labels in test_loader:
        predictions = model(inputs)
        # ... calculate accuracy ...
```

Saving and Loading Models

We don't want to retrain our model every time.

The recommended way is to save only the learned parameters (the “state dictionary”)

```
# --- SAVING ---  
# Assume 'model' is our trained model object  
torch.save(model.state_dict(), 'model_weights.pth')  
  
# --- LOADING ---  
# 1. First, we must re-create the model's structure  
model_new = MySimpleNet(10, 20, 2)  
  
# 2. Then, load the saved weights into the structure  
model_new.load_state_dict(torch.load('model_weights.pth'))  
model_new.eval()
```

Alternative: Saving the Entire Model

As an alternative, we can save the entire model object.

This saves the model structure and the weights together.

- Pro: Very simple.
- Con: Less flexible. The saved object depends on the original class definition and file structure

This command saves the entire model object.

```
torch.save(model, 'model.pth')
```

This command loads the entire model back into memory.

```
model = torch.load('model.pth')
```

In-Class Practice (Submit to Canvas)

Your goal is to build, train, and evaluate a neural network that learns a pre-defined function.

- Step 1: Generate Data. Generate a dataset (e.g., 100 samples) from the function:

$$y = 3x_1 + 4x_2 - x_3^2$$

- Step 2: Construct Model. Construct a neural network to predict y given x . You will need to customize the depth (layers) and width (neurons).
- Step 3: Train Model. Train your network on the data. Monitor the training loss and observe how it changes over time.
- Step 4: Evaluate Model. Generate a new set of test data and evaluate your model's performance on it.

Requirement:

- Implement your network based on `nn.Module`.
- Save the model and load it with another script file to make predictions.



Dataset & Dataloader

The Problem with Loading Data

- We can't load a 1TB dataset into memory at once.
- We need a way to load data from disk in small **batches**.
- We need to **shuffle** the data every epoch.
- We might want to use multiple CPU cores to **load data in parallel** so the GPU isn't waiting.

Solution: Dataset and DataLoader

- PyTorch provides two elegant classes to solve this.

- A class for storing and preprocessing data samples.
- It provides an interface for accessing and manipulating data.

Built-in Datasets

- torchvision (CV)
- torchtext (NLP)
- torchaudio (Audio)

```
from torchvision import datasets
from torchvision.transforms import ToTensor
train_dataset = datasets.FashionMNIST(root='data', download=True,
                                     train=True, transform=ToTensor())
test_dataset = datasets.FashionMNIST(root='data', download=True, train
                                     =False, transform=ToTensor())
```

It wraps a Dataset to enable efficient and flexible data loading.

- Main Purpose: Iterate over the dataset, **returning batches of data.**
- It handles complex operations for us:
 - Sampling from the dataset
 - Collating data into a batch
 - Prefetching batches for acceleration
 - Parallel processing (num_workers)
 - Shuffling
 - Distributed training support

Using the Dataloader

Create a Dataloader: You pass the Dataset object and define batch parameters

```
from torch.utils.data import DataLoader

# (train_dataset and test_dataset from previous slide)

train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=
    True, num_workers=10)
test_dataloader = DataLoader(test_dataset, batch_size=64, shuffle=
    False, num_workers=10)
```

Iterate over batches

```
for X, y in test_dataloader:
    print(X.shape, y.shape)

torch.Size([64, 1, 28, 28]) torch.Size([64])
torch.Size([64, 1, 28, 28]) torch.Size([64])
...
```

Custom Datasets

- inherit `torch.utils.data.Dataset`
- Must implement two methods:
 - `__init__(self, ...)`: Load data (e.g., read CSV)
 - `__len__(self)`: Return dataset size
 - `__getitem__(self, idx)`: Return the idxth sample

Titanic Dataset

- Use Pclass, Fare, Age, SibSp, Parch as features, Survived as label;
- Use the last 100 samples as the test set

```
from torch.utils.data import DataLoader, Dataset
import pandas as pd
import torch
class TitanicDataset(Dataset):
    def __init__(self, path, train=True):
        super().__init__()
        titanic_df = pd.read_csv(path).fillna(0)
        if train:
            self.data = titanic_df.iloc[:-100]
        else:
            self.data = titanic_df.iloc[-100:]
        self.X = self.data[['Pclass', 'Fare', 'Age', 'SibSp', 'Parch']]
        self.y = self.data['Survived'].tolist()
    def __len__(self):
        return len(self.y)
    def __getitem__(self, index):
        return torch.tensor(self.X[index], dtype=torch.float32), \
            torch.tensor(self.y[index], dtype=torch.float32)
train_dataset = TitanicDataset('data/titanic.csv', train=True)
train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=True)
```

- Create a neural network that can process the 5 numerical features (Pclass, Fare, Age, SibSp, Parch) from the Titanic dataset and predict survival.
- Train your network using your training data.
- Evaluate the test loss.

Note: Use `nn.Sigmoid()` as the final activation function.

Full Workflow: CIFAR-10 Image Classifier

Let's put all concepts together to build a complete image classifier.

- Dataset: Use `'torchvision.datasets.CIFAR10'`.
- Dataloader: Wrap it in a `'DataLoader'`.
- Module: Define a Convolutional Neural Network (CNN) using `'nn.Module'`.
- Autograd: Define a loss (`'nn.CrossEntropyLoss'`) and optimizer (`'optim.SGD'`).
- Training: Run the full training loop.

1. Load Data

```
import torchvision
import torchvision.transforms as transforms

# Images are 3-channel (RGB), 32x32 pixels
# We normalize them to the range [-1, 1]
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

trainset = datasets.CIFAR10(root='./data', train=True, download=True,
                             transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                           shuffle=True, num_workers=2)

testset = datasets.CIFAR10(root='./data', train=False, download=True,
                             transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
                                          shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')
```

2. Define the CNN Model

```
import torch.nn as nn
import torch.nn.functional as F
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        # 3 color channels, 6 output channels, 5x5 kernel
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120) # 5x5 is the image dim
            after 2 pools
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10) # 10 classes

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # Flatten all dims except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
net = Net().to(device)
```

3. Define Loss and Optimizer

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

4. Train the Network

```
net.train() # Set model to training mode
for epoch in range(2): # loop over the dataset multiple times
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data[0].to(device), data[1].to(device)

        # 1. zero the parameter gradients
        optimizer.zero_grad()
        # 2. forward + backward
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        # 3. optimize
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999: # print every 2000 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 2000:.3f}')
            running_loss = 0.0
    print('Finished Training')
```

5. Test the Network

```
correct = 0
total = 0
net.eval() # Set model to evaluation mode
with torch.no_grad(): # We don't need gradients for testing
    for data in testloader:
        images, labels = data[0].to(device), data[1].to(device)

        # calculate outputs by running images through the network
        outputs = net(images)

        # the class with the highest energy is what we choose as
        # prediction
        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()
print(f'Accuracy of the network on the 10000 test images: {100 *
      correct // total} %')
```

Deep Learning and PyTorch

- the need for deep learning frameworks

Tensors in PyTorch

- attributes, creating, reshaping, moving tensors to GPU
- tensor Operations: element-wise, broadcasting, in-place, indexing/slicing, concatenation, stack, split, aggregation, linear algebra

Autograd & Model Building

- computation graph, loss functions, backpropagation and autograd, optimizer
- building models with nn.Module
- model.train(), model.eval(), torch.no_grad()
- saving and loading models

Dataset & Dataloader

- build-in dataset, custom dataset

Python Programming with Artificial Intelligence

Thank you for listening!

